



Interrupt Implementation

Recap of Interrupt Basics

- Allow CPU to stop its current task and respond to an external event
- This avoids polling, where CPU would waste time checking for events continuously

Example: Instead of the CPU constantly checking if a sensor has data, it can do other tasks and handle the sensor data only when interrupted

Interrupt Flow

1. Interrupt occurs
2. CPU Saves State
3. Interrupt Handled
4. Resume original task (unless its a hard fault)

Handling Interrupts

Interrupt Requests (IRQ): Signals the CPU that an interrupt is ready to be handled. The signal is a physical voltage change

- **Interrupt Line:** Each interrupt has a dedicated line or wire

Interrupt Vector Table:

- Array of function pointers that map interrupts to specific memory addresses
- Cortex M4 starts this table at address `0x0`
- These are often pre-configured in the setup code and rarely need manual config

Example: You won't need to write an interrupt handler from scratch.

- Pre-made function names are used

Exception number	IRQ number	Offset	Vector
16+n	n	0x0040+4n	IRQn
.	.	.	.
.	.	.	.
.	.	.	.
18	2	0x004C	IRQ2
17	1	0x0048	IRQ1
16	0	0x0044	IRQ0
15	-1	0x0040	Systick
14	-2	0x003C	PendSV
13		0x0038	Reserved
12			Reserved for Debug
11	-5	0x002C	SVCall
10			Reserved
9			
8			
7			
6	-10	0x0018	Usage fault
5	-11	0x0014	Bus fault
4	-12	0x0010	Memory management fault
3	-13	0x000C	Hard fault
2	-14	0x0008	NMI
1		0x0004	Reset
		0x0000	Initial SP value

Efficiency and Best Practices

- **Keep Interrupt Service Routines ISRs Short:** ISRs should be minimal and fast for several reasons:
 - i. Return to normal operation quickly
 - ii. Minimize delay for other interrupts
 - iii. Less code means fewer bugs
 - iv. Assembly code is hard; focus on keeping logic in simpler C functions

Example A SysTick interrupt routine that only updates a counter instead of handling complex logic

- **Reentrant Functions:** Functions called within an ISR should be reentrant, meaning they do not rely on shared global/static variables
 - **Non-Reentrant Example:** A swap function using a global tmp variable would fail if two ISRs call it simultaneously
 - **Fix:** Use local variables instead of shared ones to ensure thread safety

Interrupt Priorities

- **Priority System:** Lower numbers indicated higher priority interrupts. In Cortex M4, some interrupts (e.g. Non-Maskable Interrupts - NMIs) always have the highest priority and cannot be changed
- **Nested Interrupts:** Some system can handle one interrupt while another is running

Interrupt Masking

- **Maskable Interrupts (IRQs):** These can be temporarily disabled or ignored based on the system needs (non-critical)
 - **Example:** A timer interrupt might be disabled when a critical real-time task is running
- ****Non Maskable Interrupts (NIMs):** Cannot be turned off and handle critical events such as hard fault failures

- **Example:** The CPU halts for NMI, ensuring immediate attention to critical issues

Type	Exception Number	IRQ Number	Exception Type	Priority	Handled Using
processor core exceptions or internal interrupts	1	n/a	Reset	-3	
	2	-14	NMI	-2	
	3	-13	HardFault	-1	
	4	-12	MemManage	Configurable	Fault handlers
	5	-11	BusFault		
	6	-10	UsageFault		
	11	-5	SVCall		System handlers
	14	-2	PendSV		
15	-1	SYSTic			
device-specific exceptions or external interrupts	16	0	IRQ	ISRs	
	17	1			
	...	...			

Interrupt States

Interrupts can be in several states:

1. **Pending:** Interrupt conditions met, but ISR not yet run
2. **Active:** ISR is running
3. **Pending & Active:** Next interrupt is pending while current one is being handled
4. **Inactive:** No interrupt pending or active

Example: If multiple interrupts occur close together, some might be pending while others are actively being processed, creating a potential delay in response

Global Interrupt Enable

- A single setting enable or disables all interrupts. If interrupts are disabled, they go into pending state and will be processed when re-enables
 - **Does not apply to NIMs**

Return from Interrupt (RTI Instruction)

- A special instruction `rti` (Return From Interrupt) is used to resume the original task after handling an interrupt
- Normal return statements do not work here due to the complexity of saving and restoring CPU states

Final Concepts

- **Efficiency Consideration:** When hardware interrupts occur, the ISR might only need to notify a user program to proceed. Writing simple ISRs minimizes system delay and complexity